Chapter 13 — Speculative Decoding and Mixture of Experts

> *"Speculative decoding exploits the fundamental asymmetry of verification versus generation. Verifying K tokens costs no more than generating one. Mixture of Experts decouples quality from compute cost. Together they represent the two most impactful inference-time architecture innovations of the last three years."*

Chapter Overview

LLM decode is sequential by nature: token N cannot be generated until token N-1 exists. This sequential dependency is what makes decode memory-bandwidth-bound and difficult to parallelize. Speculative decoding breaks this constraint by exploiting an asymmetry that has always existed but was not previously leveraged: *verifying K tokens in one forward pass costs the same as generating one token*. A cheap draft model proposes K candidates; the expensive target model verifies all K simultaneously. The result: 2–3.6× TPOT reduction at no quality cost.

Mixture of Experts decouples a different constraint: model quality scales with parameter count, but inference compute cost scales with active parameter count. MoE lets a 671B parameter model run at the compute cost of a 37B dense model while delivering quality that dense models require 200B+ parameters to match.

This chapter builds the engineering foundation for both techniques — the mathematics, the infrastructure requirements, the production failure modes, and the decision framework for deploying each.

Sections in this chapter:

- 13.1 Why Decode Is Sequential — And Why That Asymmetry Can Be Exploited
- 13.2 Speculative Decoding — Algorithm, Math, and Variants
- 13.3 MoE Architecture — Dense vs Sparse, Router Mechanics, Load Balancing
- 13.4 MoE Infrastructure Challenges — Expert Parallelism and All-to-All
- 13.5 Production Deployment — Speculative Decoding
- 13.6 Production Deployment — MoE Serving
- 13.7 Combining Speculative Decoding and MoE
- 13.8 Decision Framework and Key Numbers
- 13.9 Key Takeaways and Review Questions

13.1 Why Decode Is Sequential — And the Asymmetry That Breaks It

13.1.1 The Sequential Dependency

```
DECODE SEQUENTIAL DEPENDENCY:

  Standard autoregressive decode (LLaMA-3 70B, batch=1):

  Step 1: context → GPU → token_101  (GPU busy: 42ms)
  Step 2: context + token_101 → GPU → token_102  (GPU busy: 42ms)
  Step 3: context + tokens_101-102 → GPU → token_103  (GPU busy: 42ms)
  ...

  Total time for 100 tokens: 100 × 42ms = 4.2 seconds
  GPU utilization during decode:
    Model weights: 140 GB must be loaded from HBM for each step
    FLOPs per step: 2 × 70B × 1 token = 140 GFLOPs
    AI: 140G / 140GB = 1.0 FLOP/byte → deeply memory-bandwidth-bound
    Tensor Core utilization: ~0.6% (the GPU is 99.4% idle on compute)

  THE PROBLEM: The GPU is nearly always waiting for data, not computing.
  Adding more Tensor Cores does not help. The bottleneck is HBM bandwidth
  and the hard sequential dependency between steps.

THE VERIFICATION ASYMMETRY:

  Generating 1 token at position N requires:
    - One forward pass through 70B model
    - Time: ~42ms (memory-bound, loads 140GB model)

  Verifying K tokens simultaneously at positions N+1 to N+K requires:
    - ONE forward pass through 70B model (processes K tokens in parallel)
    - Time: ~42ms for small K (same memory-bound cost — loads 140GB model once)

  THIS IS THE KEY ASYMMETRY:
    Generation:   1 token costs T
    Verification: K tokens cost T  (same cost, K tokens checked at once!)

  If verification is free, and draft tokens can be obtained cheaply,
  the expected throughput is:
    speedup ≈ E[accepted tokens per iteration] / 1 token

  Speculative decoding makes this concrete.
```

13.2 Speculative Decoding — Algorithm, Math, and Variants

13.2.1 The Full Algorithm

```
SPECULATIVE DECODING — COMPLETE ALGORITHM:

Inputs:  Target model M_p (large, expensive: 70B)
         Draft model  M_q (small, fast: 7B)
         Prompt prefix, K (draft tokens per iteration)

LOOP:

  PHASE 1 — DRAFT (small model, K sequential steps):
    draft_tokens = []
    draft_probs  = []

    for i in range(K):
      q_dist = M_q(prefix + draft_tokens)   # 7B forward pass
      t_i    = sample(q_dist)               # sample next token
      draft_tokens.append(t_i)
      draft_probs.append(q_dist[t_i])       # store probability assigned

    Time: K × T_q = K × c × T_p   (c = T_q / T_p ≈ 0.10 for 7B/70B)

  PHASE 2 — VERIFY (large model, ONE forward pass over K+1 positions):
    all_probs = M_p(prefix + draft_tokens)   # 70B forward pass, K+1 tokens
    Time: 1 × T_p  (same as generating a single token)

  PHASE 3 — ACCEPT/REJECT (each draft token independently):
    accepted = []
    for i in range(K):
      p_i = all_probs[i][draft_tokens[i]]  # Target model prob for draft token i
      q_i = draft_probs[i]                  # Draft model prob for same token

      accept_prob = min(1.0, p_i / q_i)    # Acceptance criterion

      if random() < accept_prob:
        accepted.append(draft_tokens[i])    # Keep draft token
      else:
        # Reject: sample from residual distribution (ensures M_p's distribution)
        residual = max(0, all_probs[i] - q_probs[i])
        residual = residual / sum(residual)
```

```
        accepted.append(sample(residual))
        break                              # Reject all subsequent drafts

    # If all K accepted: take the (K+1)th token from target model
    if len(accepted) == K:
      accepted.append(sample(all_probs[K]))

    prefix = prefix + accepted

CORRECTNESS GUARANTEE:
  The acceptance criterion min(1, p_i/q_i) ensures:
    When p_i ≥ q_i: accept with probability 1.0 (target agrees or prefers this
token)
    When p_i < q_i: accept with probability p_i/q_i (partial acceptance)

  The residual sampling on rejection restores the exact target distribution.
  Mathematical proof: the marginal distribution of each accepted token
  exactly equals the distribution that M_p would produce autoregressively.

  SPECULATIVE DECODING IS NOT AN APPROXIMATION.
  It is an exact algorithm. Output quality is IDENTICAL to standard decoding.
  No accuracy evaluation needed before deployment.
```

13.2.2 Speedup Math

```
SPEEDUP FORMULA DERIVATION:

Let:
  α = acceptance rate per token (P(accept draft_i) averaged over steps)
  K = draft tokens proposed per iteration
  c = T_q / T_p = draft forward pass time / target forward pass time

Expected tokens accepted per iteration:
  E[accepted] = Σ_{i=0}^{K-1} [i × P(first reject at position i)] + K × P(all accepted)

  Simplified form (uniform acceptance rate α):
    E[accepted] = (1 - α^{K+1}) / (1 - α)

  Examples:
    α=0.6, K=5: E[accepted] = (1-0.6^6) / (1-0.6) = 0.953/0.4 = 2.38
    α=0.8, K=5: E[accepted] = (1-0.8^6) / (1-0.8) = 0.738/0.2 = 3.69
    α=0.9, K=5: E[accepted] = (1-0.9^6) / (1-0.9) = 0.469/0.1 = 4.69
    α=0.95, K=5: E[accepted] = 4.91
```

```
Time per iteration:
  T_iter = K × T_q + T_p = (K × c + 1) × T_p

Speedup vs standard (1 token per T_p):
  Speedup = E[accepted] / T_iter × T_p
          = E[accepted] / (K × c + 1)

NUMERICAL SPEEDUP TABLE (c=0.10, meaning draft is 10× faster than target):

  α\K      K=3    K=5    K=8    K=12
  ──────────────────────────────────────
  α=0.60   1.52×  1.55×  1.52×  1.47×
  α=0.70   1.73×  1.83×  1.82×  1.77×
  α=0.80   1.96×  2.26×  2.36×  2.34×
  α=0.85   2.08×  2.51×  2.70×  2.72×
  α=0.90   2.22×  2.94×  3.29×  3.41×
  α=0.95   2.37×  3.43×  4.10×  4.49×
  ──────────────────────────────────────

KEY INSIGHTS:
  1. Speedup is most sensitive to acceptance rate α, not to K.
     Improving α from 0.7 → 0.9 gives +70% more speedup at K=5.
     Doubling K from 5 → 12 at α=0.8 gives only +3% more speedup.
     Focus engineering effort on increasing α, not K.

  2. There is an optimal K for each α:
     At high α (>0.9): large K is beneficial (many tokens get accepted)
     At low α (<0.6):  K=3 is often better (less draft overhead)

  3. BREAKEVEN: speculative decoding is profitable when:
     E[accepted] > 1  →  α > 1/(K+1)
     For K=5: α > 1/6 = 0.167 → almost always profitable
     Even α=0.2 with K=5: E[accepted] = 1.04 → 1.04× speedup (tiny but positive)

  4. WHEN SPEC DECODE IS NOT WORTH IT:
     Large batch sizes (batch > 32): target model is compute-bound.
       Adding draft overhead does not proportionally increase accepted tokens.
       The verification step's cost doubles at batch=32 (K+1 tokens × batch).
     Highly creative outputs (code with novel structure, creative writing):
       α typically drops to 0.40-0.55 → 1.3-1.5× speedup, may not justify
complexity.
```

13.2.3 Acceptance Rate by Workload Type

```
EMPIRICAL ACCEPTANCE RATES (LLaMA-3 8B draft → LLaMA-3 70B target):

  Task Category              α range      Production speedup
  ──────────────────────────────────────────────────────────

  SQL generation             0.87-0.93    3.0-3.5×
  Code completion (Python)   0.82-0.90    2.8-3.3×
  Structured output (JSON)   0.85-0.95    3.0-3.6×
  Factual Q&A                0.75-0.85    2.4-3.0×
  Summarization              0.72-0.82    2.2-2.7×
  Multi-turn chat             0.68-0.80    2.0-2.5×
  Creative writing            0.55-0.70    1.5-2.0×
  Low-temperature (T=0)      0.85-0.95+   3.0-4.0× (greedy: near-perfect)
  High-temperature (T=1.2)   0.50-0.65    1.4-1.9×
  ──────────────────────────────────────────────────────────


KEY OBSERVATION:
  Temperature is the strongest predictor of acceptance rate.
  Low temperature (greedy/top-1): draft and target almost always agree → α > 0.9
  High temperature (creative): more randomness → less agreement → α drops to 0.55

WORKLOAD-ADAPTIVE STRATEGY:
  Different request types warrant different K:
    Code/SQL (α > 0.85):  K=8, high speedup
    Chat (α ≈ 0.75):      K=5, moderate speedup
    Creative (α < 0.60):  K=3, or disable spec decode

  In vLLM: route by system_prompt or model parameter to different spec-decode
configs.
```

13.2.4 Speculative Decoding Variants

```
VARIANT COMPARISON:

METHOD 1: SEPARATE DRAFT MODEL (standard)
  Draft: LLaMA-3 8B (16 GB FP8 weights)
  Target: LLaMA-3 70B (70 GB FP8 weights)
  Tokenizer: MUST BE IDENTICAL (same vocabulary, same tokenizer)
  Speedup: 2.4-3.5× (K=5, α=0.75-0.92)
  Memory: 86 GB total (fits on H200, tight on H100)

  TOKENIZER REQUIREMENT — NON-NEGOTIABLE:
```

```
    Draft and target must share identical vocabulary and tokenizer.
    If tokenizer differs: draft token IDs do not map to same vocabulary.
    Accepted tokens would silently be different words → garbage output.
    This is why Mistral 7B is the canonical draft for Mixtral 8×7B
    (same SentencePiece tokenizer, identical vocabulary).
    NOT because Mistral 7B is architecturally similar —
    because it shares the tokenizer.

METHOD 2: EAGLE (Eagle-2 Speculative Sampling, 2024)
  Approach: Train a small auto-regressive head on top of target model's hidden
states.
  No separate model: the EAGLE head is 1-2 layers using the target's features
directly.
  Draft accuracy: typically higher than separate 7B model (has access to target
features).
  Speedup: 3.0-3.5× (comparable to or better than separate draft model)
  Memory overhead: ~0.5-1 GB (just the EAGLE head weights, not a full model)
  Requirement: EAGLE weights must be trained for each target model variant.
  vLLM support: speculative_model="yuhuili/EAGLE-LLaMA3.1-Instruct-70B"

METHOD 3: NGRAM SPECULATION (prompt lookup)
  Approach: Look for repeated n-grams in the input prompt.
  No model at all: uses the prompt itself as the draft source.
  Use case: tasks where output repeats or paraphrases the prompt (RAG,
summarization).
  Speedup: 1.3-1.7× (limited by n-gram match rate)
  Memory: zero additional memory
  vLLM: speculative_model="[ngram]", ngram_prompt_lookup_max=4

METHOD 4: LAYER SKIPPING
  Draft: early exit after first N/2 layers of the target model
  Target: full N layers for verification
  Speedup: 1.5-1.8× (draft shares the same weights — no extra memory)
  Quality: lower than separate draft (draft quality limited by early exit)
  Use when: no small model available and memory is extremely constrained

VARIANT SELECTION GUIDE:
  Small draft model available AND fits in memory: Method 1 (best quality)
  Best quality + low memory overhead: Method 2 EAGLE
  Zero setup, code/RAG workloads: Method 3 NGram
  Absolute zero extra memory: Method 4 Layer Skipping
```

13.2.5 Production Configuration (vLLM)

```python
```

```python
from vllm import LLM, SamplingParams

# ── Method 1: Separate draft model ──────────────────────────────────────
llm = LLM(
    model="meta-llama/Llama-3.3-70B-Instruct",
    speculative_model="meta-llama/Llama-3.2-1B-Instruct",  # Same tokenizer!
    num_speculative_tokens=5,              # K = 5 draft tokens
    speculative_draft_tensor_parallel_size=1,   # Draft on 1 GPU
    tensor_parallel_size=2,                # Target on 2 GPUs (TP=2 for 70B)
    gpu_memory_utilization=0.90,
    quantization="fp8",
)

# ── Method 2: EAGLE ─────────────────────────────────────────────────────
llm = LLM(
    model="meta-llama/Llama-3.3-70B-Instruct",
    speculative_model="yuhuili/EAGLE-LLaMA3.1-Instruct-70B",
    num_speculative_tokens=5,
    tensor_parallel_size=2,
)

# ── Method 3: NGram speculation ─────────────────────────────────────────
llm = LLM(
    model="meta-llama/Llama-3.3-70B-Instruct",
    speculative_model="[ngram]",
    ngram_prompt_lookup_max=4,
    num_speculative_tokens=5,
    tensor_parallel_size=2,
)

# Sampling: no changes needed; spec decode is transparent to the caller
params = SamplingParams(temperature=0.8, top_p=0.95, max_tokens=512)
outputs = llm.generate(prompts, params)

# ── Monitoring acceptance rate ──────────────────────────────────────────
# vLLM Prometheus metric: vllm:spec_decode_draft_acceptance_rate
# Target: > 0.75 for meaningful speedup
# If < 0.60: consider different draft model, reduce K, or disable spec decode
import requests
r = requests.get("http://localhost:8000/metrics")
# Look for: vllm:spec_decode_draft_acceptance_rate
```

```
# If metric shows 0.45 for a batch of requests → spec decode is net negative:
# overhead exceeds benefit → disable for that workload tier
```

13.3 Mixture of Experts — Architecture and Theory

13.3.1 Dense vs Sparse Models: The Fundamental Trade-off

```
DENSE vs MOE — THE CORE COMPARISON:

Dense model (LLaMA-3 70B):
  Every token activates EVERY parameter in every layer
  Compute per token ∝ total_params = 70B
  Quality ∝ total_params = 70B
  Memory ∝ total_params = 140 GB (BF16)

MoE model (DeepSeek-V3 671B total, 37B active):
  Every token activates only K selected experts per FFN layer
  Compute per token ∝ active_params = 37B   ← much cheaper!
  Quality ∝ total_params = 671B              ← much better!
  Memory ∝ total_params = 671B               ← much larger!

MoE EQUATION:
  Dense FFN:   output = W₂ · GELU(W₁ · x)             (one FFN)
  MoE FFN:     output = Σᵢ∈topK(x) gᵢ(x) · Expertᵢ(x) (K of N experts)

  Where:
    Expertᵢ = independent FFN with its own W₁ᵢ, W₂ᵢ weights
    gᵢ(x)   = gating weight from router (softmax output, renormalized)
    topK     = indices of K highest-scoring experts for this token

REAL MODEL DATA:

  Model            Total params   Active params  Active/Total  MMLU
  ──────────────────────────────────────────────────────────────────
  LLaMA-3 70B      70B (dense)    70B            100%          80.3%
  Mixtral 8×7B     47B            13B            28%           70.6%
  Mixtral 8×22B    141B           39B            28%           77.8%
  DeepSeek-V3      671B           37B            5.5%          88.5%
  GPT-4 (est)      ~1.8T          ~220B (est)    ~12%          86.4%
  ──────────────────────────────────────────────────────────────────
```

```
KEY INSIGHT:
  DeepSeek-V3 (671B total, 37B active) runs at the compute cost of a 37B
  dense model while delivering better quality than any 200B dense model.
  This is the MoE value proposition: decouple quality from compute cost.

  MEMORY IMPLICATION:
    Quality comes from TOTAL parameters (all must be stored).
    Compute comes from ACTIVE parameters (only these run per token).
    Memory = f(total). Compute = f(active). These scale independently.
    DeepSeek-V3 needs 671B × 1B = ~671 GB FP8 weights.
    This requires 9+ H100s just for weights, despite 37B active compute.
```

13.3.2 Router Mechanics

```
ROUTER ARCHITECTURE (per MoE layer):

Input: token hidden state x ∈ ℝ^{d_model}

Step 1: Compute expert scores
  scores = Softmax(W_router · x)
  W_router ∈ ℝ^{N × d_model}  (N = total experts)
  scores ∈ ℝ^N, each ∈ (0,1), sum = 1.0

Step 2: Select top-K experts
  top_k_idx = argsort(scores)[-K:]         # K highest scores
  top_k_w   = scores[top_k_idx]
  top_k_w   = top_k_w / sum(top_k_w)       # Renormalize to sum = 1

Step 3: Execute selected experts and combine
  output = Σᵢ∈top_k_idx  top_k_w[i] × Expertᵢ(x)

TYPICAL CONFIGURATIONS:
  Model         N (experts)  K (active)  Sparsity (K/N)
  ──────────────────────────────────────────────────────
  Mixtral 8×7B  8            2           25.0%
  Mixtral 8×22B 8            2           25.0%
  DeepSeek-V3   64 routed +  6 + 2       9.4% routed
                2 shared     always      + 2 shared
  ──────────────────────────────────────────────────────


DEEPSEEK-V3 SHARED EXPERT INNOVATION:
  2 experts are ALWAYS activated for EVERY token (shared experts).
  6 additional experts are selected from 64 routed experts.
```

```
  Total active: 8 experts per token per MoE layer.

  Purpose of shared experts:
    Learn universal patterns that apply to all tokens.
    Provide stable gradient signal independent of routing decisions.
    Prevent the "expert collapse" failure mode (see §13.3.3).
```

13.3.3 Load Balancing — The Critical Training Problem

```
EXPERT COLLAPSE WITHOUT LOAD BALANCING:

  Step 1:  Random init → Expert 3 slightly better initialized
  Step 2:  Router learns to send slightly more tokens to Expert 3
  Step 3:  Expert 3 receives more gradient signal → improves faster
  Step 4:  Router, now correctly reasoning, sends MORE to Expert 3
  Step 5+: Positive feedback loop → 80-95% of tokens → Expert 3
           Other experts see no gradient → remain untrained → wasted capacity

  Final state: equivalent to a 1-expert model. Zero benefit from MoE.
  This is why EVERY MoE model requires explicit load balancing during training.

LOAD BALANCING TECHNIQUE 1 — AUXILIARY BALANCE LOSS (standard):

  L_balance = α × N × Σᵢ (fᵢ × Pᵢ)

  Where:
    fᵢ = fraction of tokens routed to expert i in current batch (measured)
    Pᵢ = mean router probability for expert i (differentiable)
    α  = balance loss coefficient (typically 0.01-0.1)
    N  = number of experts

  Effect: When fᵢ is high (expert overloaded), Pᵢ is likely high too.
          Product fᵢ × Pᵢ is high → L_balance penalizes this → loss function
          pushes router away from concentrating on any single expert.

  Tuning α: too small → imbalance persists; too large → model quality degrades.

LOAD BALANCING TECHNIQUE 2 — EXPERT CAPACITY FACTOR:

  capacity_i = ceil((B × S × K / N) × capacity_factor)

  Where B=batch, S=seq_len, K=active experts, N=total experts.
```

```
  capacity_factor = 1.0: Strict; no expert can get more tokens than average
  capacity_factor = 1.25: 25% overflow buffer (most common in production)

  Tokens exceeding capacity: "overflow tokens"
    Option A: Drop (fast, slight quality loss — common in training)
    Option B: Route through residual connection only (safe fallback)
    Option C: Route to next-best expert (most correct, adds compute)

  Production recommendation: capacity_factor=1.25, Option B overflow handling.

LOAD BALANCING TECHNIQUE 3 — DEEPSEEK-V3 AUXILIARY-LOSS-FREE:

  Instead of a separate loss term, use per-expert bias terms b_i:
    scores_biased = scores + b_i  (bias added before top-K selection)
    Expert routing uses biased scores; expert output uses unbiased scores

  Update rule for b_i:
    If expert i received > target_fraction of tokens this step: decrease b_i
    If expert i received < target_fraction this step: increase b_i

  Effect: Dynamic adjustment that steers routing toward uniform distribution
  WITHOUT adding noise to the training objective.
  Advantage: Primary loss function is uncontaminated by balancing terms.
  Result: Near-perfect load balance with better training stability than loss-
based.
```

13.4 MoE Infrastructure Challenges

13.4.1 Expert Parallelism — The New Parallelism Dimension

```
EXPERT PARALLELISM (EP):

  Definition: Distribute different experts across GPUs.
  Each GPU holds E/EP experts.
  When a token is routed to an expert on a different GPU:
    → Send the token to that GPU for expert computation
    → Receive the computed result back

  This requires ALL-TO-ALL communication — not AllReduce!

EP CONFIGURATIONS (Mixtral 8×7B, 8 experts):
```

```
  EP=1:  All 8 experts on 1 GPU (large GPU required: 47B BF16 = 94 GB)
  EP=2:  4 experts per GPU (2 GPUs; within NVLink is fine)
  EP=4:  2 experts per GPU (4 GPUs)
  EP=8:  1 expert per GPU (8 GPUs; enables H100 SXM deployment)

TYPICAL COMBINED PARALLELISM FOR MOE:
  Mixtral 8×7B on 8× H100 SXM:
    TP=1 (per-GPU expert, no tensor parallelism within expert)
    EP=8 (one expert per GPU)
    Attention layers: replicated (small, fits on each GPU)
    Expert layers: distributed (one expert per GPU)
```

13.4.2 All-to-All Communication — Mechanics and Cost

```
ALL-TO-ALL COMMUNICATION PATTERN (EP=8, one expert per GPU):

Step 1 — ROUTING DECISION (local, no communication):
  Each GPU: run router on its token batch
  GPU 0 determines: "token A → Expert 3 (GPU 3), token B → Expert 7 (GPU 7)"
  GPU 1 determines: "token C → Expert 1 (GPU 1), token D → Expert 5 (GPU 5)"
  ...

Step 2 — DISPATCH (All-to-All round 1):
  Every GPU sends token embeddings to every other GPU based on routing.
  GPU 0 sends: tokens_for_GPU3 → GPU 3, tokens_for_GPU7 → GPU 7, etc.
  GPU 1 sends: tokens_for_GPU0 → GPU 0, tokens_for_GPU4 → GPU 4, etc.
  Communication pattern: N×N matrix of variable-size messages.

  Message sizes vary by token distribution (not fixed like AllReduce).

Step 3 — EXPERT COMPUTE (local, no communication):
  Each GPU executes its expert on all received tokens.
  Full FFN GEMM on the received batch.

Step 4 — COMBINE (All-to-All round 2):
  Send computed results back to originating GPU.
  Same volume as dispatch (same tokens, different direction).

Total All-to-All volume per MoE layer:
  V = 2 × B × S × K × d_model × bytes_per_element

  (2 = dispatch + combine, K = active experts per token)
```

```
NUMERICAL EXAMPLE (batch=32, seq=4096, K=2, d_model=4096, BF16):
  V = 2 × 32 × 4096 × 2 × 4096 × 2B = 8.6 GB per MoE layer

COMPARISON TO TP ALLREDUCE:
  TP AllReduce: 2 × B × S × d_model × 2B = 4.3 GB per layer (for same batch)
  MoE All-to-All: 8.6 GB per layer (2× TP, because K=2)
  AND MoE All-to-All is harder to overlap with compute than TP AllReduce.

ALL-TO-ALL BANDWIDTH AND INTERCONNECT:
  Unlike TP AllReduce (latency-critical → must use NVLink),
  All-to-All is bandwidth-bound — InfiniBand is acceptable.

  Intra-node All-to-All (NVLink, EP≤8): latency ~1-2ms
  Cross-node All-to-All (IB NDR 400G):  latency ~5-10ms

  Production rule: keep EP ≤ 8 (within one NVLink domain) when possible.
  When EP > 8: cross-node All-to-All adds 3-5× latency vs intra-node.
```

13.4.3 Expert Load Imbalance in Production

```
EXPERT LOAD IMBALANCE — THE PRODUCTION FAILURE MODE:

  Even with training-time load balancing, production workloads can
  trigger imbalance that was not present in training data:

  SCENARIO:
    EP=8, 1 expert per GPU
    Mean tokens per expert per step: 100 (perfectly balanced)
    During a burst of SQL queries: Expert 4 receives 450 tokens
    Other experts: 30-70 tokens each

  WHY THIS HURTS:
    All-to-All in EP: ALL GPUs must finish their expert compute BEFORE
    the combine All-to-All can proceed (barrier synchronization).
    GPU 4 (Expert 4): runs for 450ms on its large batch
    GPU 0-3,5-7:      finish in 70ms and WAIT for GPU 4

    System throughput = throughput of the SLOWEST expert per step.

    With 4.5× imbalance: effective throughput ≈ 1/4.5 = 22% of balanced.
    This is a 78% throughput loss from load imbalance alone.

MONITORING FOR EXPERT IMBALANCE:
```

```
  Metric: expert_load_variance = std(tokens_per_expert) / mean(tokens_per_expert)
  Healthy: < 0.20 (all experts within 20% of mean)
  Warning: 0.20-0.50
  Critical: > 0.50 (some expert getting 3× or more)

  Per-expert token count histogram (log in production):
    prometheus.Histogram("moe_expert_token_count", buckets=[50,100,200,400,800])

  Alert rule:
    ANY expert receiving > 2× mean token count for > 5 consecutive steps.

MITIGATION STRATEGIES:
  At training time (best fix):
    Increase auxiliary loss weight α (forces more uniform distribution).
    Add capacity factor overflow handling (prevents any expert getting unlimited
tokens).
    Use auxiliary-loss-free method (DeepSeek approach, better stability).

  At deployment time (when retraining is not possible):
    Increase capacity_factor from 1.25 to 1.5 (more overflow handling).
    Route overflow tokens to residual instead of dropping.
    Apply request-routing to avoid all SQL/code queries hitting same server.

  Monitoring as mitigation (early detection):
    The expert_load_variance metric should be graphed per step in Grafana.
    Any increase in variance > 30% vs baseline: paged as P1 alert.
    Pattern of consistently high variance: retrain with higher α.
```

13.5 Production Deployment — Speculative Decoding

13.5.1 Memory Budget and Hardware Selection

```
SPECULATIVE DECODING MEMORY REQUIREMENTS:

  Config: LLaMA-3 70B (FP8) + LLaMA-3 8B draft (FP8)

  Target model: 70 GB (FP8)
  Draft model:   8 GB (FP8)
  KV cache (target, S=4096): ~10 GB on H100 (after weights)
  KV cache (draft): proportional to 8B, much smaller (~1.2 GB)
```

```
  Total: 70 + 8 + 10 + 1.2 = 89.2 GB → does NOT fit on H100 (80 GB)!

  HARDWARE OPTIONS:
    1× H200 (141 GB): 141 - 89.2 = 51.8 GB spare → fits comfortably
    2× H100 TP=2:     160 - 89.2 = 70.8 GB spare → fits with TP=2
    1× MI300X (192 GB): fits easily, 102 GB spare for KV

  With INT4 target (35 GB) + FP8 draft (8 GB):
    1× H100: 80 - 35 - 8 - 10 - 1.2 = 25.8 GB for KV → fits!
    Best option for cost-sensitive deployment.

DRAFT ON SEPARATE GPU:
  Alternative: run draft model on a dedicated GPU.
  Target and draft communicate activation tensors between each iteration.
  Benefit: draft GPU is fully dedicated → no memory pressure on target.
  Cost: 1 extra GPU (justified only at very high concurrency).
  Latency: draft-target communication adds ~1ms per iteration.
```

13.5.2 When to Enable and When to Disable

```
SPECULATIVE DECODING DECISION MATRIX:

  Condition                                Enable?   Notes
  ──────────────────────────────────────────────────────────────────────────
  α > 0.75 (code, SQL, structured)         YES       2.5-3.5× speedup
  α 0.65-0.75 (factual, chat)              YES       1.8-2.4× speedup, worth it
  α 0.50-0.65 (creative, high temp)        MAYBE     1.3-1.6×, evaluate overhead
  α < 0.50                                 NO        Net negative (draft overhead >
benefit)
  Batch size > 32                          NO        Target becomes compute-bound;
                                                      draft overhead not amortized
  Memory pressure (< 15 GB headroom)    NO        Use EAGLE or NGram instead
  Low temperature (T=0, greedy)            YES       Highest speedup (α near 1.0)
  Very long outputs (> 1000 tokens)        YES       Fixed overhead amortized over
many iters
  Real-time streaming (P99 TPOT SLA)    YES       Reduces TPOT directly
  ──────────────────────────────────────────────────────────────────────────

AUTOMATIC FALLBACK WHEN α DROPS:
  Monitor: vllm:spec_decode_draft_acceptance_rate (rolling 1-minute window)
  If rate < 0.60 for > 2 minutes:
    Disable spec decode for new requests in this window.
    Re-enable after rate recovers above 0.70 for 5 minutes.
```

```
  This prevents cases where a traffic shift (e.g., from SQL to creative writing)
  turns spec decode into a net overhead.
```

13.6 Production Deployment — MoE Serving

13.6.1 Memory and Parallelism Configuration

```
MOE MODEL DEPLOYMENT CONFIGURATIONS:

MIXTRAL 8×7B (47B total, 13B active):
  BF16 weights: 47B × 2B = 94 GB
  FP8 weights:  47B × 1B = 47 GB

  H100 options:
    BF16: 2× H100 TP=2 (160 GB total, 94 GB weights, 66 GB KV/KV)
    FP8:  1× H100 (80 GB total, 47 GB weights, 33 GB KV)
    FP8 + EP=8: 8× H100 (1 expert per GPU + TP attention)
      → Attention replicated, experts distributed
      → All-to-All within NVLink domain: fast

DEEPSEEK-V3 (671B total, 37B active):
  FP8 weights: 671B × 1B = 671 GB
  Minimum GPUs: ceil(671/80) = 9 H100s (tight, no KV headroom)
  Production: 16× H100 TP=2, EP=8 (per MoE layer)
              FP8 + INT8 KV to maximize concurrent sessions

  Configuration breakdown:
    TP=2: attention layers distributed across 2 GPUs (within NVLink)
    EP=8: each H100 pair holds ~84 GB FP8 expert weights
          plus attention weights replicated at TP=2
    Total: 16 GPUs for 671B model + reasonable KV headroom

VLLM CONFIGURATION FOR MIXTRAL:
  python -m vllm.entrypoints.openai.api_server \
    --model mistralai/Mixtral-8x7B-Instruct-v0.1 \
    --tensor-parallel-size 2 \
    --quantization fp8 \
    --max-model-len 32768 \
    --gpu-memory-utilization 0.90
```

13.6.2 All-to-All Monitoring in Production

```
MOE PRODUCTION MONITORING METRICS:

  Metric                            Target      Alert Threshold
  ──────────────────────────────────────────────────────────────
  moe_expert_token_count (per exp) ≤1.5× mean  > 2.0× mean
  moe_alltoall_latency_ms          < 3ms (NVLink) > 6ms
  moe_capacity_overflow_fraction   < 0.01        > 0.05
  moe_tokens_dropped_fraction      < 0.001       > 0.01
  expert_load_variance (CV)        < 0.20        > 0.50
  ──────────────────────────────────────────────────────────────

  Prometheus example:
    histogram_quantile(0.99,
      rate(moe_expert_token_count_bucket[1m])) /
    avg(rate(moe_expert_token_count_sum[1m]))
    > 2.0
    → Alert: expert load imbalance

  Grafana dashboard essential panels:
    1. Per-expert token count heatmap (time × expert_id)
    2. All-to-All latency P99 over time
    3. Overflow fraction per MoE layer
    4. Total MoE throughput vs theoretical (active param compute)
```

13.7 Combining Speculative Decoding and MoE

```
SPEC DECODE + MOE INTERACTION:

  CHALLENGE 1: Draft model selection
    MoE target (e.g., Mixtral 8×7B): requires tokenizer-identical draft.
    Best choice: Mistral 7B as draft for Mixtral 8×7B.
    WHY MISTRAL 7B SPECIFICALLY:
      - Same SentencePiece tokenizer as Mixtral family
      - Same vocabulary (32,000 tokens)
      - NOT because it's architecturally similar
    Cannot use: Llama-3 7B (different tokenizer), Gemma 7B (different vocab)

  CHALLENGE 2: Verification cost with MoE
    Verification = ONE forward pass through full target model at batch=K+1
```

```
    For MoE: this means full expert routing for K draft tokens simultaneously

    At K=5: verification processes 5 tokens through the target model.
    This is like a prefill batch of 5 — still memory-bound (AI ≈ 5 FLOP/byte).
    The All-to-All dispatch still occurs for these 5 tokens.

    Net effect on speedup:
      MoE verification is slightly more expensive than dense (All-to-All overhead)
      Practical speedup slightly lower: ~2.1× vs ~2.5× for equivalent dense

    Example (Mixtral 8×7B + Mistral 7B, K=5, α=0.75):
      Standard Mixtral decode: ~900 tok/s (8-GPU EP=8)
      With spec decode: ~1,900 tok/s (2.1× speedup)

  CHALLENGE 3: KV cache
    MoE does not change the attention layer structure.
    KV cache for MoE = same as equivalent dense model with same d_model, heads, L.
    PagedAttention and all KV optimizations apply unchanged.
    No MoE-specific KV changes needed.

  EXPERT ROUTING FOR DRAFT TOKENS:
    The draft tokens (K tokens in a batch) are routed through the MoE experts.
    Routing for K tokens can be unbalanced (K=5 may all go to Expert 3).
    For such small batches: individual step imbalance is irrelevant.
    Production imbalance is measured over hundreds of batches, not individual
steps.
```

13.8 Decision Framework and Key Numbers

13.8.1 Deployment Decision Tree

```
OPTIMIZATION DECISION TREE:

┌──────────────────────────────────────────────────────────────────────────┐
| GOAL: Reduce TPOT (decode latency) without hardware upgrade               |
|                                                                          |
| Is batch size small (< 32)?                                              |
|   YES → Speculative decoding is applicable                               |
|   NO  → Target is compute-bound; spec decode less effective              |
|                                                                          |
| What is the workload type?                                               |
```

```
│   Code/SQL/structured (α > 0.80) → Standard spec decode, K=5-8 (3-3.5×)│
│   Chat/factual (α 0.65-0.80)     → Spec decode K=5 (2-2.5×)           │
│   Creative/high-temp (α < 0.60)  → NGram spec (1.3-1.7×) or disable   │
│                                                                        │
│ Memory headroom available?                                             │
│   > 10 GB spare → Separate draft model (best quality)                 │
│   2-10 GB spare → EAGLE (low overhead, good quality)                  │
│   < 2 GB spare → NGram or layer-skipping                              │
└────────────────────────────────────────────────────────────────────────┘


┌────────────────────────────────────────────────────────────────────────┐
│ GOAL: Scale model quality without scaling inference compute             │
│                                                                        │
│ Training a new model?                                                  │
│   YES → MoE architecture (2-5× quality per active FLOP)               │
│   NO  → Cannot retrofit existing dense models                          │
│                                                                        │
│ Deploying an existing MoE model?                                       │
│   Step 1: Match EP to available GPUs per NVLink domain (EP ≤ 8)       │
│   Step 2: Enable FP8 quantization (expert weights are BW-bottlenecked) │
│   Step 3: Monitor expert load balance continuously                     │
│   Step 4: Set capacity_factor=1.25, overflow=residual                 │
└────────────────────────────────────────────────────────────────────────┘


┌────────────────────────────────────────────────────────────────────────┐
│ GOAL: Maximum combined throughput improvement                          │
│                                                                        │
│ Combined stack (all compatible):                                       │
│   FP8 weights:              2× decode throughput (bandwidth reduction) │
│   FP8 KV cache:             2× concurrent users                       │
│   Spec decode (code tasks): 3-3.5× TPOT improvement                  │
│   Prefix caching:           60-80% TTFT reduction                     │
│   P/D disaggregation:       ~30% additional system throughput         │
│                                                                        │
│ Combined realistic gain vs BF16 baseline: 5-10×                        │
└────────────────────────────────────────────────────────────────────────┘
```

13.8.2 Key Numbers Reference Card

```
SPECULATIVE DECODING NUMBERS:

  Typical α range (code/SQL):     0.85-0.95
  Typical α range (chat):         0.68-0.82
```

```
  Typical α range (creative):     0.50-0.65

  Speedup at α=0.80, K=5, c=0.10: 2.26×
  Speedup at α=0.90, K=5, c=0.10: 2.94×
  Speedup at α=0.95, K=5, c=0.10: 3.43×

  EAGLE overhead vs separate model: ~1 GB vs 8-16 GB
  NGram speedup (no model):        1.3-1.7×

  Breakeven α (K=5): α > 1/6 = 0.167 (always profitable unless α very low)
  Disable threshold: α < 0.55 sustained (overhead dominates)

  Draft model requirement: IDENTICAL TOKENIZER to target (mandatory)

MOE KEY NUMBERS:

  Mixtral 8×7B:    47B total, 13B active (28% sparsity), 8 experts, K=2
  Mixtral 8×22B:   141B total, 39B active, 8 experts, K=2
  DeepSeek-V3:     671B total, 37B active, 64 routed + 2 shared, K=6+2

  All-to-All volume (batch=32, S=4096, K=2, d=4096, BF16): 8.6 GB per layer
  Intra-node All-to-All latency (NVLink): 1-2ms
  Cross-node All-to-All latency (IB NDR): 5-10ms

  Expert load imbalance threshold: any expert > 2× mean → P1 alert
  Production imbalance impact: 3-5× overload → 20-40% throughput loss

  Capacity factor: 1.25 (standard) to 1.5 (conservative)
  Overflow handling: residual passthrough (preferred) or drop (fast)
```

13.9 Key Takeaways and Review Questions

```
KEY PRINCIPLES:

  1. SPECULATIVE DECODING IS EXACT, NOT APPROXIMATE:
     The acceptance criterion min(1, p/q) mathematically guarantees
     that the output distribution equals the target model's distribution.
     No quality evaluation needed before deployment.
     Safe to enable for any production workload.

  2. TOKENIZER IDENTITY IS THE ONLY HARD CONSTRAINT FOR SPEC DECODE:
```

```
     Draft and target must share identical vocabulary and tokenizer.
     Architecture differences do not matter. Quality differences do not matter.
     Tokenizer mismatch: silent output corruption (wrong semantics accepted).
     This is the #1 implementation bug in spec decode deployments.

  3. ACCEPTANCE RATE DOMINATES SPEEDUP — NOT K:
     At α=0.75 vs α=0.90 with K=5: speedup difference is 2.3× vs 2.9× (26%
better).
     At K=5 vs K=10 with α=0.75: speedup difference is 2.3× vs 2.5× (9% better).
     Optimize for workloads with high α. Don't increase K beyond 8.

  4. MOE: COMPUTE = F(ACTIVE), MEMORY = F(TOTAL):
     DeepSeek-V3 runs at 37B compute cost but requires 671B memory.
     This asymmetry defines the entire deployment challenge.
     Cannot serve a 671B MoE model on hardware designed for a 37B dense model.
     Hardware sizing must account for total parameters, not active parameters.

  5. EXPERT LOAD IMBALANCE IS THE PRODUCTION FAILURE MODE FOR MOE:
     4× imbalance → 75% throughput loss (slowest expert sets the barrier).
     It is often invisible in per-GPU utilization (all GPUs appear busy).
     Requires explicit expert_load_variance monitoring with P1 alerts.
     Training-time auxiliary loss is the only permanent fix.

  6. ALL-TO-ALL IS BANDWIDTH-BOUND, NOT LATENCY-BOUND:
     Unlike TP AllReduce (must use NVLink), MoE All-to-All tolerates IB latency.
     Prefer EP ≤ 8 (intra-node NVLink) for lowest latency.
     Cross-node EP is viable for throughput-oriented serving (not latency-
critical).
```

Review Questions

Difficulty: Foundational

Question 1 (Correctness Proof + Failure Mode): Explain why speculative decoding produces output statistically identical to standard autoregressive decoding. What would happen if the acceptance criterion were changed to `accept_prob = 1.0` always (accept all draft tokens unconditionally)?

Expected answer: The acceptance criterion min(1, p_i/q_i) works as follows: when the target model assigns higher probability than the draft model (p_i ≥ q_i), the token is always accepted (min(1, ≥1) = 1.0). When the draft model over-estimates (q_i > p_i), the token is accepted with probability p_i/q_i < 1. On rejection, the output is sampled from the residual distribution max(0, p - q) / Z, which exactly compensates for the over-confident draft tokens. Mathematically, the marginal distribution of each emitted token equals M_p's autoregressive distribution — this is

provable by showing that the expected output distribution over accepted and rejection-resampled tokens integrates to M_p.

If accept_prob = 1.0 always (unconditional acceptance): every draft token is accepted regardless of target model agreement. The output distribution would follow M_q (the draft model) rather than M_p (the target model). The quality of outputs would degrade to the draft model's quality level. This would be a silent quality regression — users would receive 7B-quality responses while paying for 70B-quality responses. This is precisely why the acceptance criterion must be probabilistic rather than always-accept.

Difficulty: Intermediate

Question 2 (Speedup Calculation for Low Acceptance Rate): A legal document summarization service uses speculative decoding with K=5 and measures α=0.55. The draft model forward pass takes c=0.10 × target model time. (a) Calculate the expected tokens per iteration and speedup. (b) Is spec decode worth deploying here? (c) What specific alternative would you recommend?

Expected answer: (a) E[accepted] = (1 - α^{K+1}) / (1-α) = (1 - 0.55^6) / (1-0.55) = (1 - 0.0277) / 0.45 = 0.972 / 0.45 = 2.16 tokens. Time per iteration = (K×c + 1) × T_p = (5×0.10 + 1) × T_p = 1.5 × T_p. Speedup = 2.16 / 1.5 = 1.44×. This is a real but modest speedup. (b) At 1.44×: spec decode is technically net positive (> 1.0×), so it does improve throughput. However, the deployment complexity (separate 7B draft model, 8-16 GB extra memory, acceptance rate monitoring, fallback logic) may not be worth 44% speedup for a legal service that likely has stricter accuracy requirements than throughput requirements. The main risk: α=0.55 means 45% of draft tokens are rejected and resampled from the residual distribution — this is a lot of rejection overhead. (c) Better alternatives: (1) NGram speculation (zero memory overhead, 1.3-1.7× speedup for summarization tasks where output re-uses input phrases) — lower speedup but near-zero operational complexity. (2) EAGLE (0.5-1 GB overhead vs 8-16 GB for separate draft, achieves α similar to or better than separate 7B model because it has access to target model features). (3) For legal summarization: primary optimization should be KV cache (long documents → prefix caching for fixed document headers/footers), not speculative decoding.

Difficulty: Principal Level

Question 3 (MoE Infrastructure Scaling): You are scaling a DeepSeek-V3-class model (671B total, 37B active, 64 experts per MoE layer, K=6, 61 MoE layers) from 1 node (8 GPUs) to a 64-GPU cluster (8 nodes). (a) Design the EP configuration. What is the maximum EP that stays within one IB switch hop? (b) Calculate the All-to-All volume per MoE layer per GPU at your chosen EP (batch=32, seq=2048, d=4096, BF16). (c) With 61 MoE layers and two All-to-Alls per layer, what is total All-to-All bandwidth required per training step at 2-second step time? (d) Is InfiniBand NDR 400G (50 GB/s per link × 8 links per node = 400 GB/s) sufficient?

Expected answer: (a) EP configuration: 64 experts across 64 GPUs = EP=64 (1 expert per GPU). Within one node (8 GPUs): EP=8 would give 8 experts per NVLink domain. For EP=64 across 8 nodes: cross-node All-to-All over InfiniBand. Maximum EP within one IB switch hop: depends on topology, but rail-optimized clusters allow 8 nodes × 8 GPUs = 64 GPUs within one fat-tree switch = EP=64 stays within one switch hop. If top-of-rack switches: EP=8 per rack stays within single switch hop. Recommend: EP=64 if all 8 nodes connect to same IB switch; otherwise EP=8 with DP=8. (b) Per GPU All-to-All volume per layer: V = 2 × (batch/EP) × seq × K × d × bytes = 2 × (32/64) × 2048 × 6 × 4096 × 2B = 2 × 0.5 × 2048 × 6 × 4096 × 2 = 2 × 0.5 × 100,663,296 = 100.7 MB per GPU per layer (each direction). Note: batch/EP = 32/64 = 0.5 — batch must be at least EP=64 for each GPU to have tokens. This reveals batch=32 is too small for EP=64! Need batch ⩾ 64. With batch=64: V = 2 × 1 × 2048 × 6 × 4096 × 2B = 201.3 MB per GPU per layer. (c) Total All-to-All per step: 61 layers × 2 All-to-Alls × 201.3 MB per GPU = 24.6 GB per GPU per step. At 2-second step time: bandwidth required = 24.6 GB / 2s = 12.3 GB/s per GPU. (d) IB NDR 400G = 50 GB/s per link × 8 links = 400 GB/s per node = 50 GB/s per GPU. Required: 12.3 GB/s per GPU. Available: 50 GB/s per GPU. IB NDR 400G provides 4× the required bandwidth. InfiniBand NDR is sufficient with 4× headroom for burst traffic and other collective operations (DP AllReduce, gradient sync).

Difficulty: Principal Level

Question 4 (MoE Expert Imbalance Diagnosis): A production Mixtral 8×7B serving cluster (EP=8, one expert per GPU) shows 34% lower throughput than theoretical. DCGM reports all 8 GPUs at 98% SM utilization, suggesting full compute saturation. Step-time analysis shows the job is completing 34% slower than expected from the arithmetic. (a) Explain the mechanism by which 98% SM utilization on all GPUs can coexist with 34% throughput loss. (b) Which specific metric would confirm expert load imbalance? (c) Provide three mitigations with tradeoffs, ordered by deployment speed.

Expected answer: (a) The key insight: 98% SM utilization means each GPU's SM is active 98% of the time — but that does NOT mean each GPU is doing equal useful work. With EP=8, after the All-to-All dispatch, each GPU executes its local expert on however many tokens it received. If Expert 4 received 4× the mean number of tokens, GPU 4 runs for 4× longer than GPU 0. GPUs 0-3,5-7 finish quickly, then their SMs spend the remainder of the step executing internal GPU operations (padding computation, waiting on the combine All-to-All barrier) or idling at the CUDA stream level while appearing "active" in SM utilization counters. SM utilization measures whether the SM issued any instruction per cycle — it does not measure whether those instructions were doing useful model work or just barrier synchronization. The 34% slowdown is caused entirely by the slowest expert (4× overloaded) gating the All-to-All combine step. (b) The diagnostic metric: per-expert token count, specifically `max(expert_token_count) / mean(expert_token_count)`. A ratio of 2.5× or higher would explain a ~30% throughput loss (the system is gated by the expert with 2.5× the tokens). In vLLM: `vllm:moe_expert_load_balance` histogram. In custom monitoring: track `tokens_dispatched_to_expert_i` for all i per step. The

34% loss corresponds to approximately: if one expert gets 3.4× the tokens, (3.4-1)/3.4 = 71% of that expert's compute is "extra" work that other GPUs wait on. With 1 of 8 experts at 3.4×: system slowdown = 3.4/8 + 7/8 = 0.425 + 0.875 → normalized ratio ≈ 1.3× slowdown... The exact math depends on the distribution, but imbalance of 3-4× explains 34% throughput loss. (c) Three mitigations (fastest deployment first): (1) Increase capacity_factor from 1.25 to 1.5 and use residual overflow (deploy in minutes, zero retraining). Expert 4 now has a higher capacity ceiling before overflowing; overflow tokens go through residual rather than waiting. Tradeoff: slightly higher memory usage, some overflow tokens get lower quality (residual vs expert). (2) Request routing: examine which request types generate the skewed traffic (likely SQL/code queries all routing through Expert 4). Route these queries to a different server instance configured with different expert placement. Tradeoff: requires traffic analysis and routing logic changes; effective but requires understanding the root cause. (3) Retrain with higher auxiliary loss weight α (weeks to months). This is the permanent fix. Increase the balance loss coefficient so the training objective more aggressively penalizes expert concentration. Tradeoff: requires full retraining, may slightly reduce model quality if α is set too high (balance at the cost of specialization quality).

Difficulty: Principal Level

Question 5 (Draft Model Architecture for Production): You are deploying speculative decoding for a multi-tenant LLaMA-3 70B serving cluster on H100 GPUs. The cluster serves mixed workloads: 60% factual Q&A (α≈0.78), 40% code generation (α≈0.88). GPU memory pressure is high — target model is FP8 (70 GB) with only 8 GB remaining on each H100 for KV cache. Design the complete deployment including which draft model, memory allocation, monitoring, and automatic fallback.

Expected answer: Memory constraint: H100 has 80 GB total. FP8 target: 70 GB. Available: 10 GB. A separate 8B FP8 draft model would need 8 GB — leaving only 2 GB for KV, which is insufficient for any meaningful serving (2 GB / 0.67 GB per 4K-context sequence = 3 sequences maximum). Conclusion: standard separate draft model does not fit on H100 with this configuration. Draft model selection: EAGLE (yuhuili/EAGLE-LLaMA3.1-Instruct-70B). EAGLE adds approximately 0.5-1 GB for the auto-regressive head weights rather than 8 GB for a full separate model. This leaves 7-9 GB for KV cache — enabling ~10-13 concurrent sequences at S=4096. Memory allocation: Target FP8: 70 GB. EAGLE head: 0.8 GB. KV budget: 80 × 0.90 - 70.8 = 1.2 GB. This is very tight. Consider using INT4 KV cache (0.335 GB/seq at S=4096): 1.2 / 0.335 = 3 sequences. Better option: reduce max_model_len to 2048 (halves KV per sequence): 0.335 GB → 3-4 sequences. Or use H200 if possible (141 GB total, leaves 70 GB for KV). The design recommendation: if the cluster is truly H100-only with FP8 target, use EAGLE with INT8 KV and max_model_len=2048. Monitoring: Track `vllm:spec_decode_draft_acceptance_rate` separately for code requests vs factual requests using request routing tags. Alert thresholds: factual queue α < 0.65 for 2 min → P2 alert; code queue α < 0.75 for 2 min → P2 alert. Combined α <

0.60 → P1 alert with auto-fallback. Automatic fallback: When combined acceptance rate drops below 0.60 for 5 consecutive minutes, disable EAGLE speculation for new requests and run standard autoregressive decode. Re-enable when rate recovers above 0.70 for 5 minutes. Implement as a feature flag in the serving layer (not in vLLM core) to avoid disrupting in-flight requests. Per-workload routing: route code requests (detected by system_prompt) to serving instances with K=8 (higher K optimal for α=0.88); route factual requests to instances with K=5. This maximizes speedup for each workload tier without sharing configurations that would compromise the tighter-α factual tier.

Further Reading

1. *Fast Inference from Transformers via Speculative Decoding* — Leviathan, Kalman, and Matias (ICML 2023). `arxiv.org/abs/2211.17192` Theorem 1 (correctness proof) and Section 4 (acceptance criterion derivation) are the mathematical foundation for everything in this chapter.
2. *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty* — Li et al. (2024). `arxiv.org/abs/2401.15077` The EAGLE auto-regressive head design. Compares to separate draft model quality.
3. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer* — Shazeer et al. (ICLR 2017). `arxiv.org/abs/1701.06538` Despite being from 2017, the load-balancing challenge and noisy top-K mechanism are directly used in every MoE model today.
4. *Mixtral of Experts* — Mistral AI (2024). `arxiv.org/abs/2401.04088` The production MoE deployment paper. Read the routing analysis and expert specialization findings (different experts handle different languages).
5. *DeepSeek-V3 Technical Report* — DeepSeek AI (2024). `arxiv.org/abs/2412.19437` Section 3 on Multi-Token Prediction training and Section 4 on auxiliary-loss-free load balancing are the current state of the art in MoE training infrastructure.
6. *Efficient Expert Parallelism for Trillion Parameter Models* — See DeepSpeed and Tutel papers. `arxiv.org/abs/2206.03382` The engineering treatment of All-to-All optimization at scale.

— End of Chapter 13 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved